

Deep Learning 3

Applications

Machine learning

Roman Jakubíček

Department of Biomedical Engineering

Brno University of Technology

Introduction

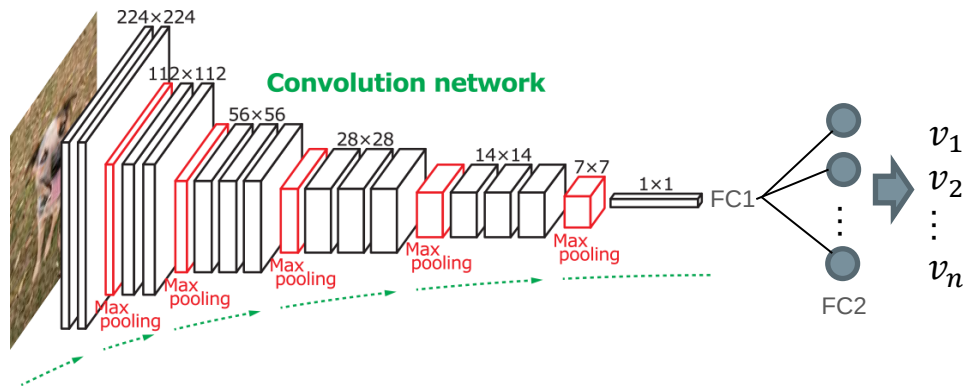
- Application of CNN
 - Classification
 - Regression
 - Image-to-image regression, segmentation
 - Object detection (R-CNN, YOLO, SSD, ...)

Regression and classification

Architecture

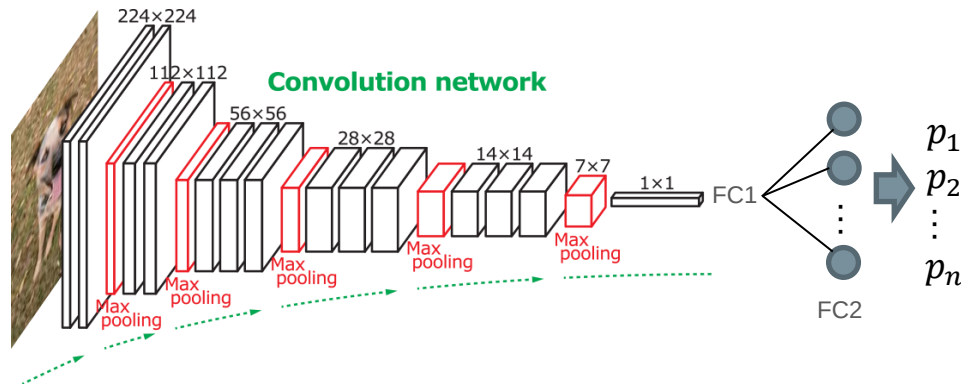
Regression problem

- Architecture: encoder => FC
- Input: image
- Output: regressed values
- Loss fcn: L_1 or L_2
- Last layer:
 - fully connected / 1x1 conv
 - no activation function (in the specific cases ReLu or softmax)
 - number of neurons corresponds to the number of regressed values
- it is sometimes appropriate that the predicted value is relative, e.g. for prediction positions,...



Classification problem

- Architecture: encoder => FC
- Input: image
- Output: vector of probabilities



- Loss fcn: Multi-class Weighted Cross-entropy
- Last layer:
 - Fully connected / 1x1 conv
 - Softmax (sigmoid)
 - Number of neurons corresponds to the number of classes

One-hot encoding

- Output vector includes values of membership probabilities for each class (length of the vector equals the number of classes)
- Resulting estimated classes is taken with maximal probabilities

Integer encoding

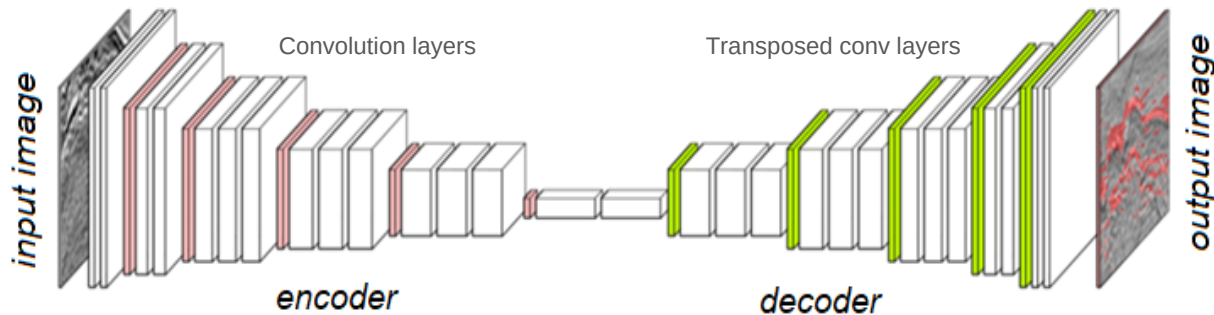
- Alternatively, the output can be integer number – a label of class (e.g. suitable for ordinal variable)

Image-to-image

Architecture

Image Regression

- Architecture: encoder – decoder
- Input: image
- Output: image
- Loss fcn: L_1 or L_2
- Last layer:
 - Convolution layer
 - Linear activation



- We need paired data (reference data for an input image) – can be problematic
- If we haven't these data, we can create e.g. synthetic or obtain during new acquisition
- There are existing special methods, which work with non-paired images

Image Regression

Utilization:

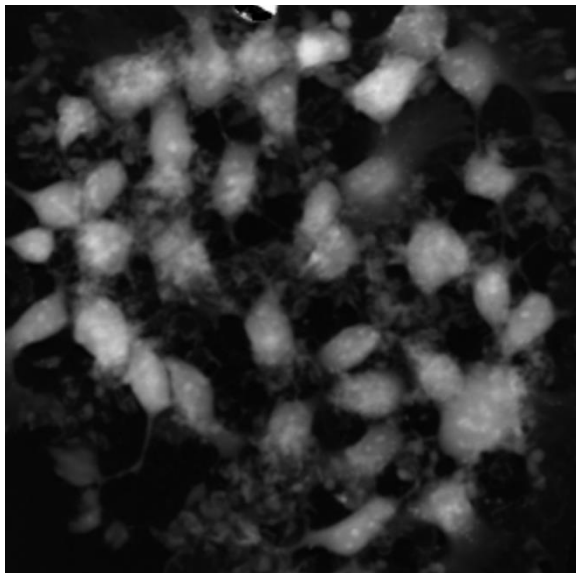
- image segmentation
- noise reduction
- reconstruction from sinogram
- image transformation to new image (GAN,...)



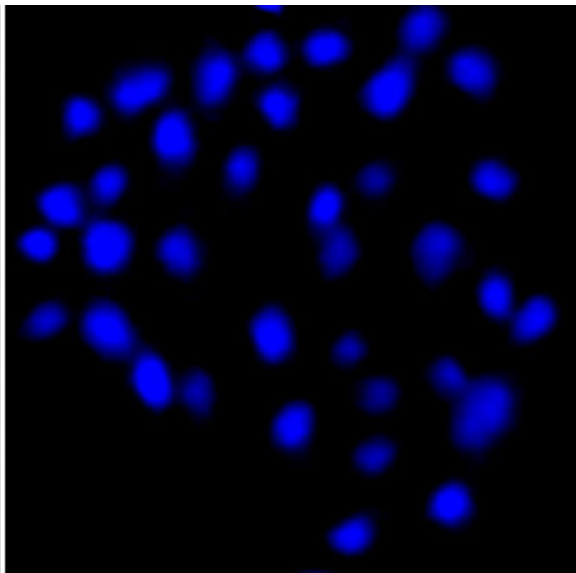
Image Regression

Example of fluorescence prediction of cells' nuclei from microscopic image

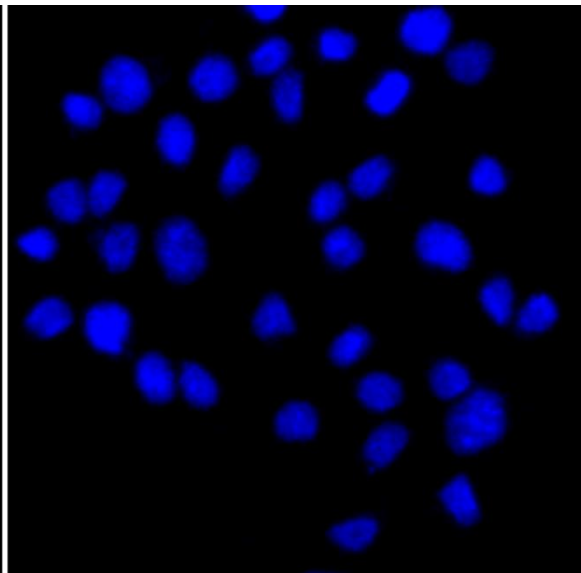
Input image



Regression image



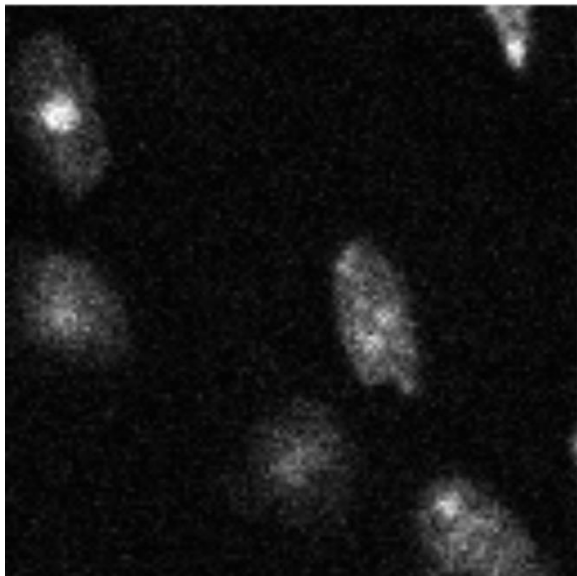
Reference image



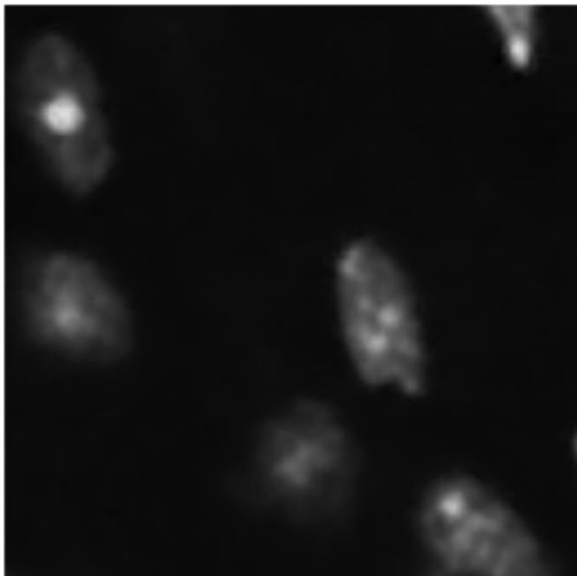
Noise reduction

Example of denoising

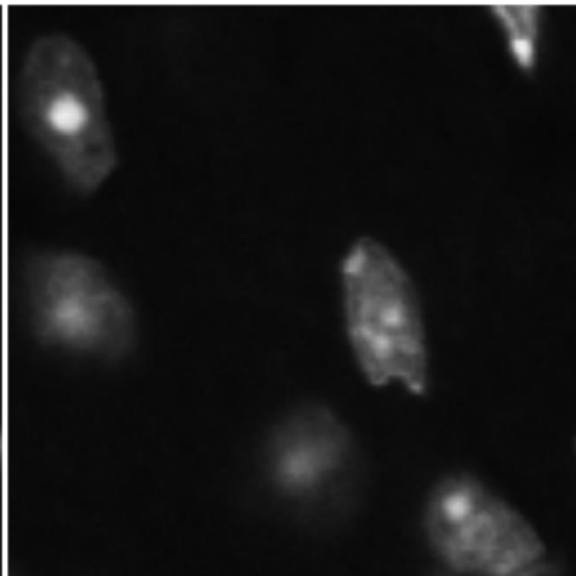
Input image



Regression image



Reference image



Next example:

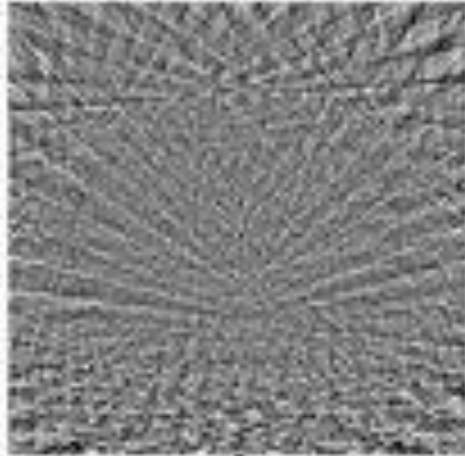
https://www.youtube.com/watch?time_continue=24&v=ooeHI_qmqzI&feature=emb_logo

Noise reduction

Input image



Regression image



Resulting image



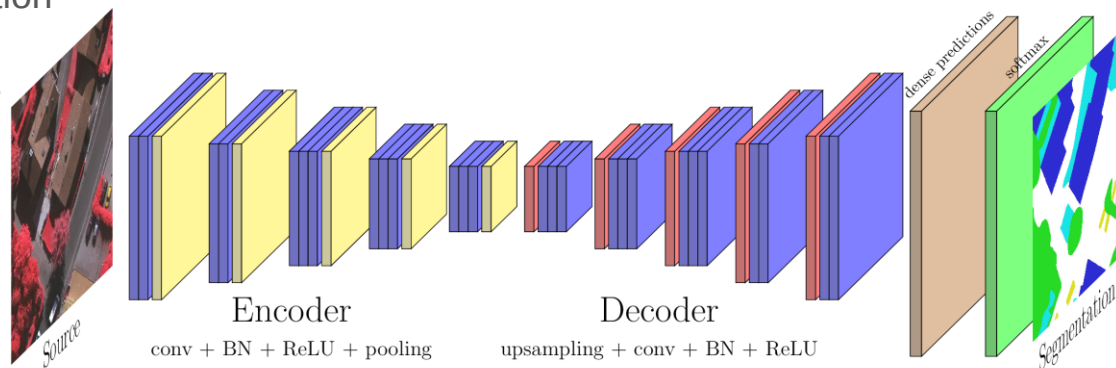
Image segmentation

A special case of the image-to-image transformation – output is a label of the classes for each pixel

- **Architecture:** encoder – decoder
- **Input:** image
- **Output:** 2D probability map for each class
- **Loss fcn:** Multi-class Weighted Cross-entropy or Binary Tverski (e.g. Dice loss)
- **Last layer:**
 - convolution layer, where its depth size = number of classes
 - softmax (through depth) for each pixel
 - alternatively 1x1 convolution

Output has size $M \times N \times D$, where D is the number of classes.

We select classes with maximal probability for each pixel.



Object detection

Architectures

Object detection

It is a computer vision technique that aims to detect objects in from bounding boxes. It can find objects such as cars, human, face or anatomical structure or pathologies.

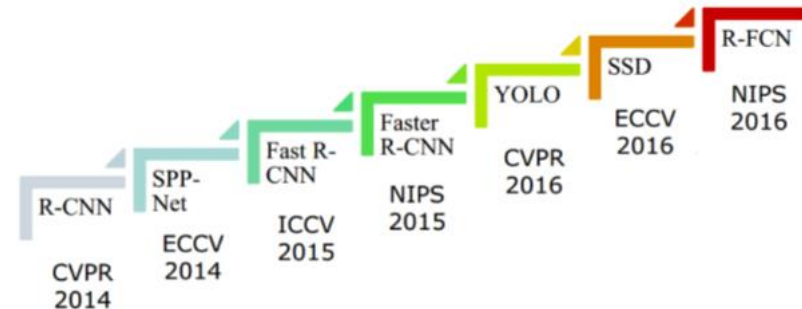
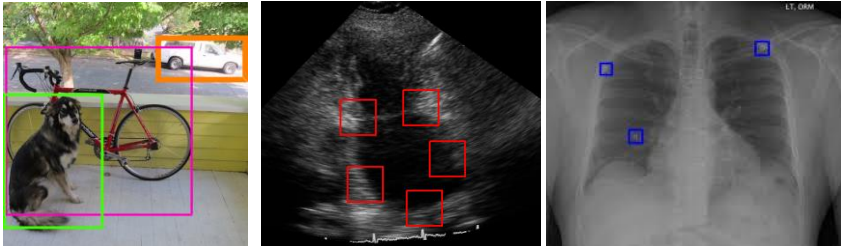
Object detection usually involves two processes:

- localization of the object
- classification of this object

Usually, these steps can be performed iteratively, parallel or in one step.

The most common algorithms:

- R-CNN (“fast”, “faster”, “cascade”)
- SSD
- YOLO (v.3)
- RetinaNet



Faster R-CNN

Region convolution neural network

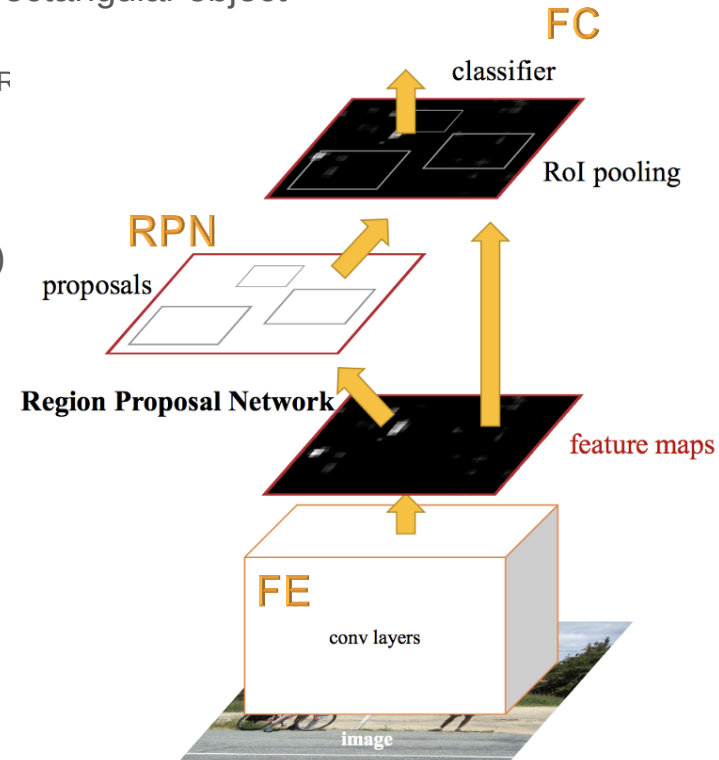
- Takes an image (of any size) as input and outputs a set of rectangular object proposals, each with an objectness score
- The third version of the algorithm (R-CNN => fast R-CNN => faster F

Basic two modules:

- extraction of features by CNN with FC classifier (fRCNN)
- network for proposal region (RPN)

- The iterative four-stage learning process
- Sharing convolution layers

Modules: FE – features extractor, FC – fully connected classificatory, RPN – region proposal network

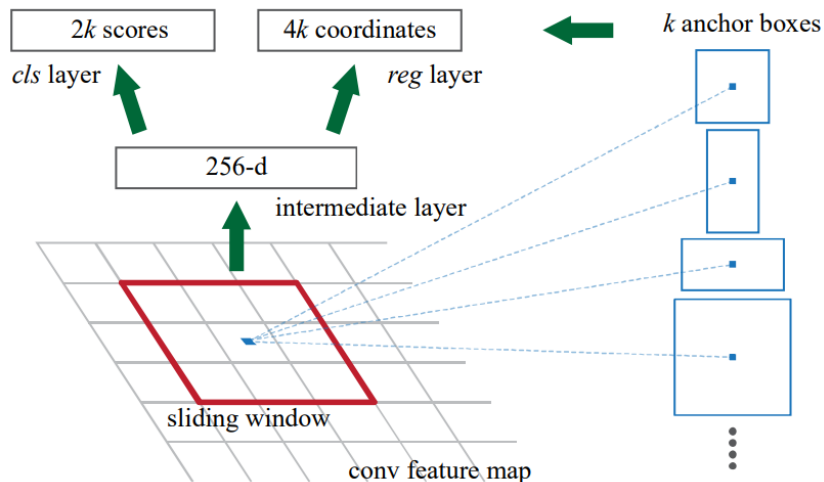


Faster R-CNN

Region convolution neural network

Region proposal network (RPN)

- 3x3 conv layer – feature maps (e.g. for VGG can have 512 maps)
- Two parallel layers:
 - regression 1x1 convolution (4k parameters – position and size of BB)
 - classification 1x1 convolution (2k parameters – values coding membership to BB/nonBB)
- The output is a set of 6k parameters for each pixel

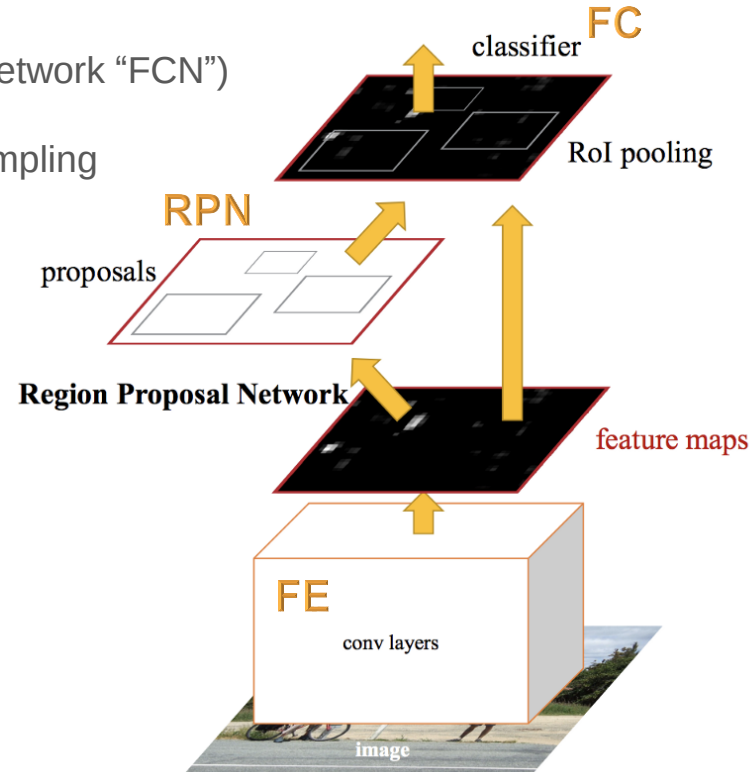
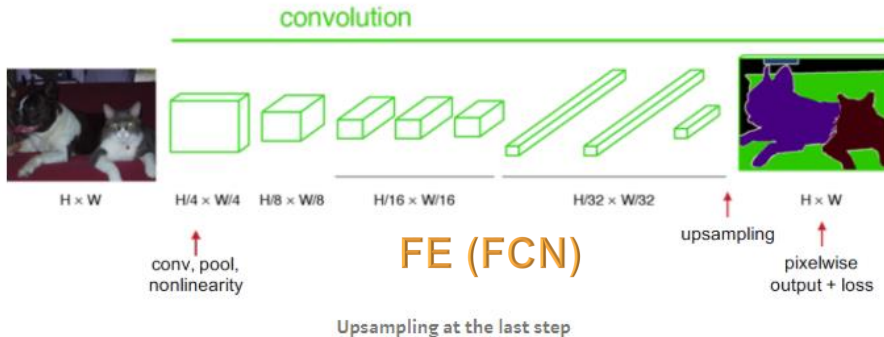


Faster R-CNN

Region convolution neural network

Detection network (FE + FC)

- Using an encoder for feature map extraction (fully conv network “FCN”)
- minimal reduction of spatial dimension
- In case of a reduction of spatial dimension is used up-sampling
- Pre-trained networks can be used (VGG, ResNet, ...)
- FC with two classification layers (two FC layers)



Faster R-CNN

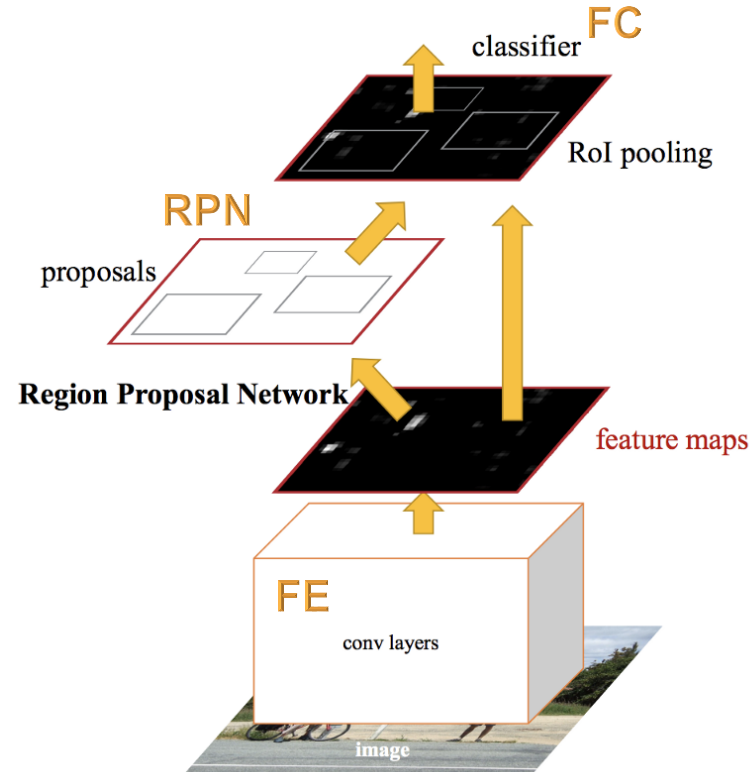
Region convolution neural network

Learning algorithm

- shared conv features
- special objective function including the reg and cls error
- optimization by backprop. and SGD
- Iterative approach

Four stages of learning:

1. RPN and FE
2. FC (with BB from RPN) and FE
3. RPN with shared and frozen layers of FE
4. FC with shared and frozen FE



Faster R-CNN

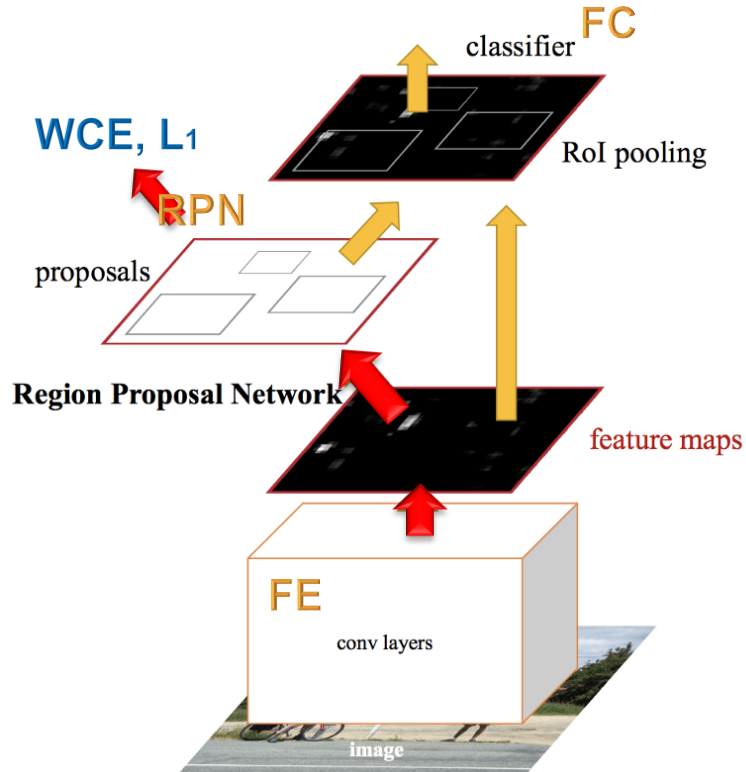
Region convolution neural network

First stage

- Learning RPN and FE
- FE uses pre-trained weights on ImageNet
- **Input of FE** – Image
- **Output of FE** – Feature maps
- **Input of RPN** – Features maps
- **Output of RPN** – positions and size of BB and their prob.

Loss functions (sum):

- classification loss (WCE)
- regression loss (L1)



Faster R-CNN

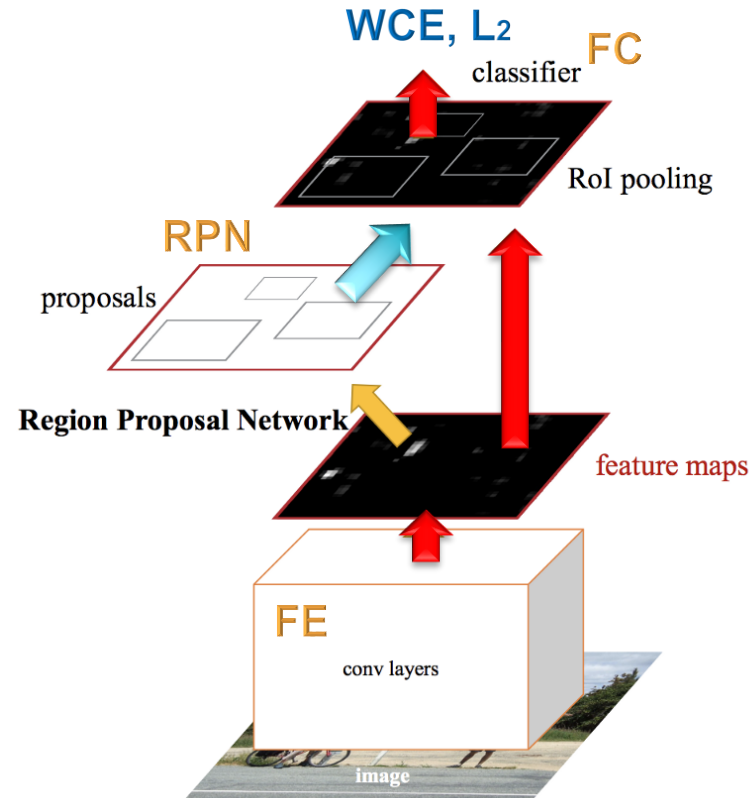
Region convolution neural network

Second stage

- Learning FE and FC
- FE uses pre-trained weights on ImageNet again as in the first stage
- **Input of FE** – Image
- **Output of FE** – Feature maps
- **Input of FC** – Features maps, BBs
- **Output of FC** – Refining of BB positions and size, and class

Loss functions (sum):

- classification loss (WCE)
- regression loss (L_2)



Faster R-CNN

Region convolution neural network

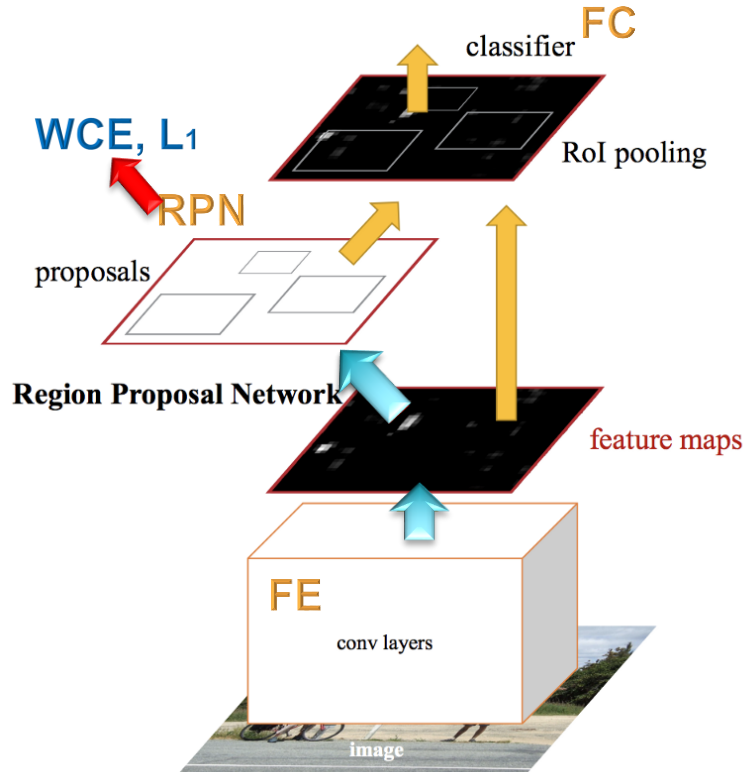
Third stage

- Learning RPN
- FE **shares** weights with FE from second stage
- **Input of FE** – image
- **Output of FE** – feature maps
- **Input of RPN** – features maps
- **Output of RPN** – positions and size of BB and their prob.

Frozen

Loss functions (sum):

- classification loss (BB or non-BB)
- regression loss (position and size of BB)



Faster R-CNN

Region convolution neural network

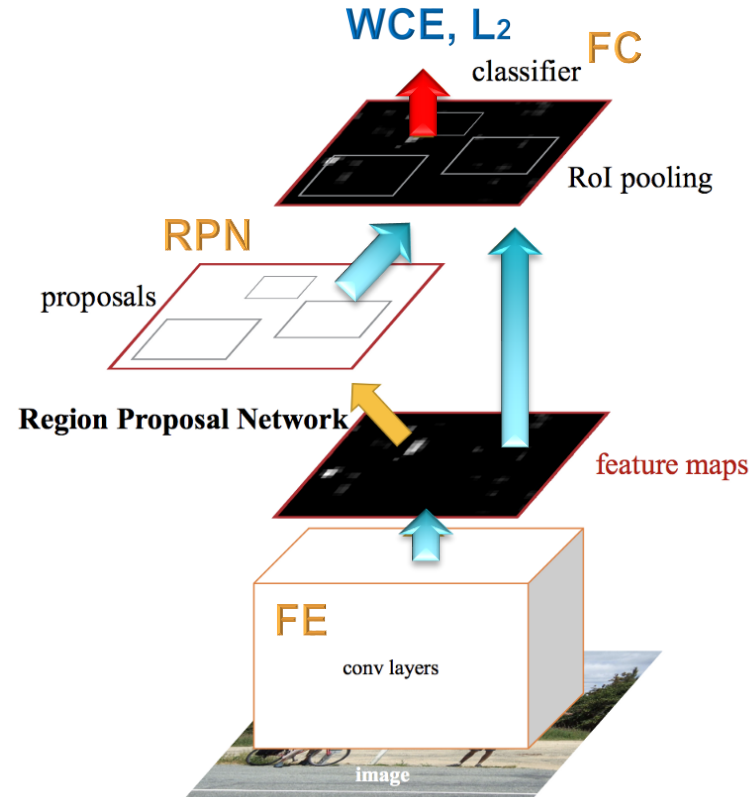
Fourth stage

- Learning FC
- FE and RPN **share** weights from third stage
- **Input of FE** – Image
- **Output of FE** – Feature maps
- **Input of FC** – Features maps, BBs
- **Output of FC** – refining of BB positions and size, and class

Frozen

Loss functions (sum):

- classification loss (WCE)
- regression loss (L_2 , refine BB position and size)



Faster R-CNN

Region convolution neural network

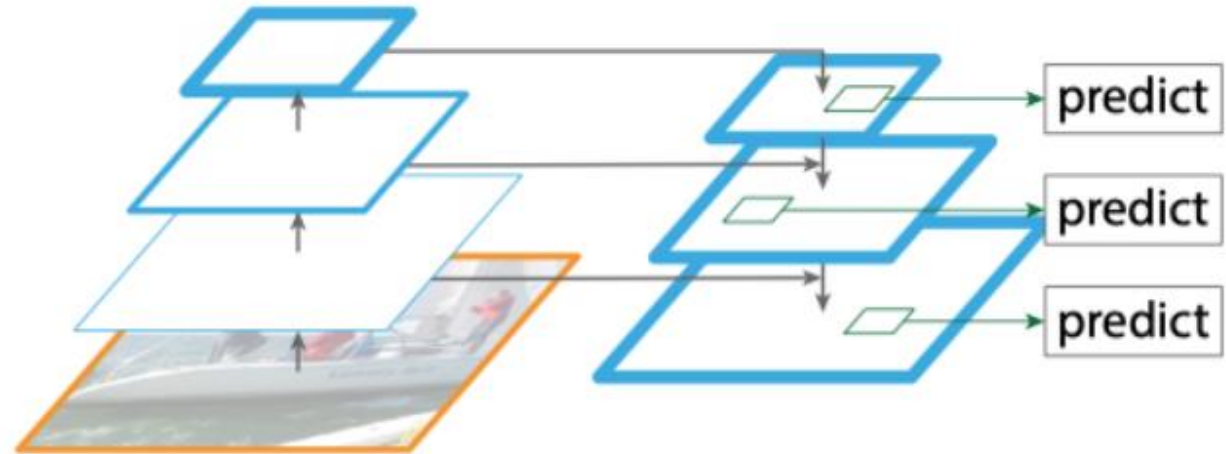
Summary

- Very fast approach
 - We can use pre-trained networks
 - objects can have different aspect ratios and sizes
 - Overlapping objects
 - The best approach so far
-
- Moreover, this architecture has been modified to “mask R-CNN” for the **segmentation problem**.

FPN

Feature pyramid networks

- well-know method in computer vision
- using features from different layers to form a feature pyramid
- basic unit of new architectures (Resnet, ...)
- difficult to share the feature pyramid between RPN and the region-based detector



YOLO *You only look once (v.3)*

- The approach involves a single neural network trained end-to-end
- Predicts bounding boxes and class labels for each bounding box directly
- Inspired by RPN and ResNet
- FE – pretrained residual networks DarkNet
- **Input:** 224x224 image
- **Output:** positions, size, and a score of BB

Process

- image is divided in first conv layer into an $S \times S$ grid
- bottlenecks are between individual blocks
- the spatial bottleneck at the end
- FC predicts $S \times S \times (5B + C)$

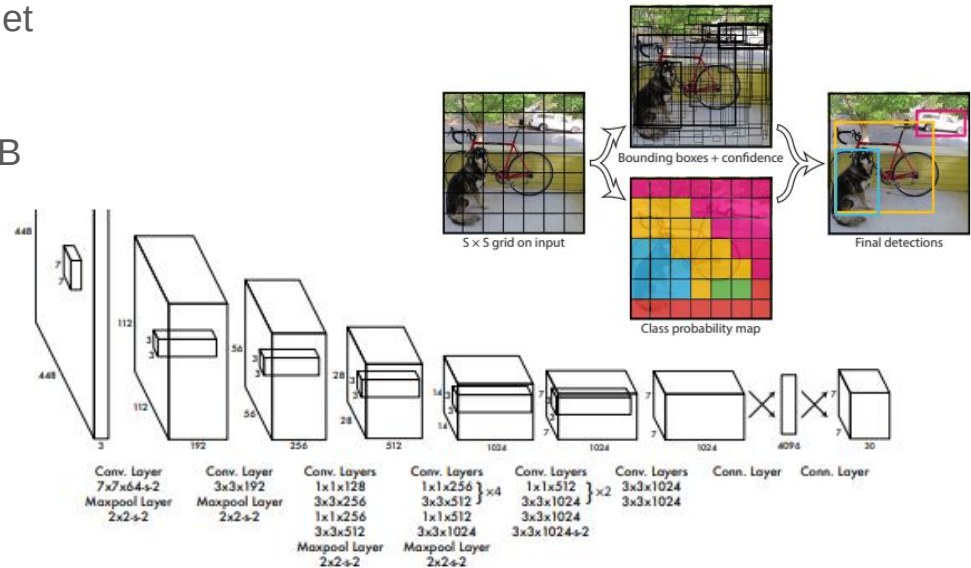


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

YOLO

You only look once

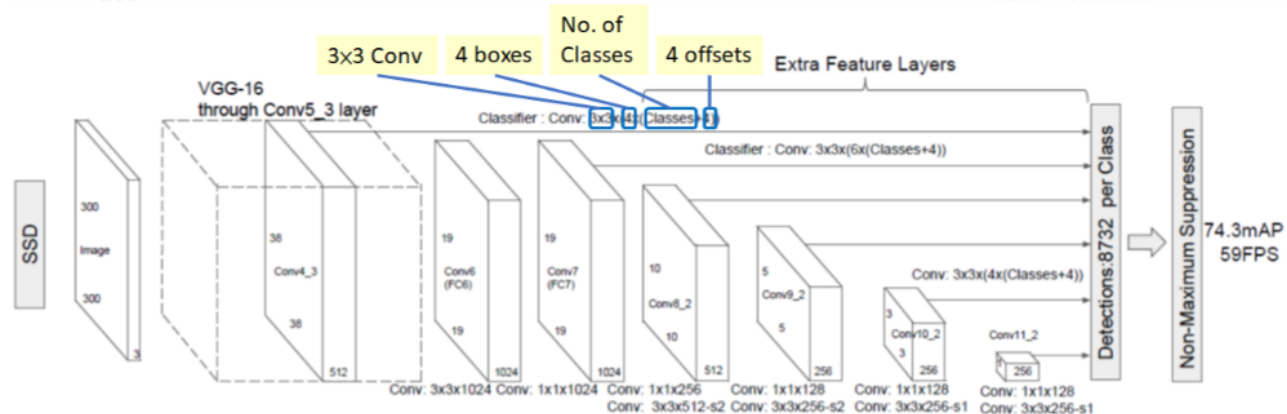
Summary

- faster but less accurate than “faster R-CNN”
- a **single convolutional network**, trained end-to-end, predicts the bounding boxes and the class probabilities for these boxes
- The newest official version v.3 (+ anchor boxes, deeper net, k-means, residual blocks)
- Unofficial v.4 or newly v.5
- **Limitations:**
 - of YOLO algorithm is that it struggles with **small objects** within the image, for example, it might have difficulties in detecting a flock of birds.
 - architecture has multiple down-sampling layers – **coarse features**

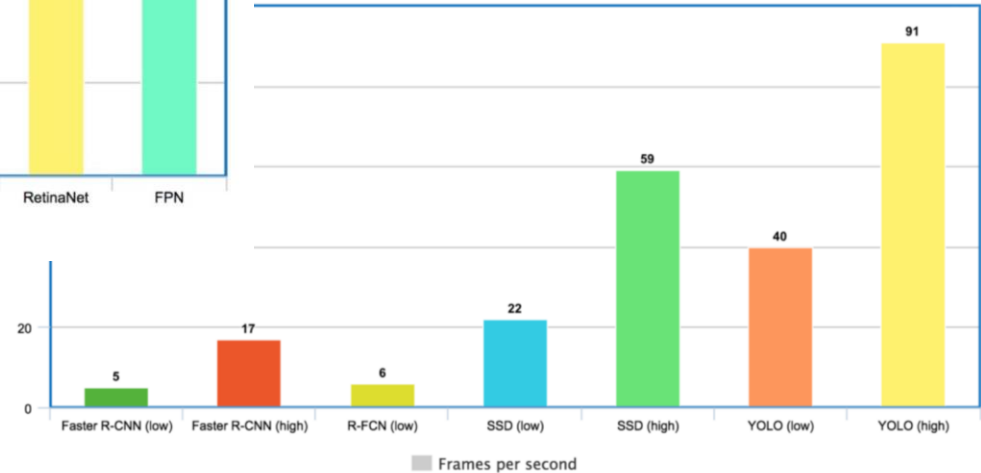
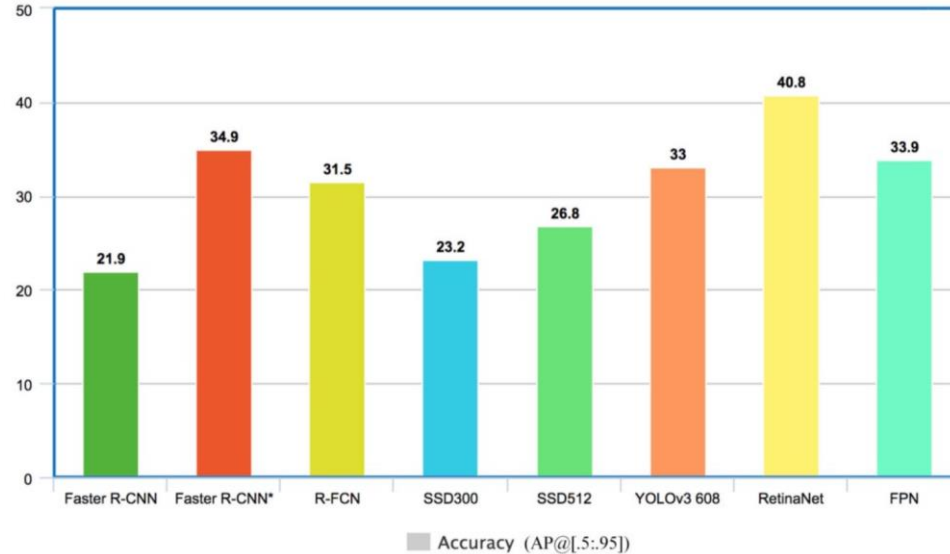
SSD

Single shot detector

- single phase with more detections
- the first part of encoder – feature extractor (part of VGG)
- the second part is a pyramidal
- special loss function
- includes the dilated convolution layers
- two types – for 300 and 512 sized image
- problem with small objects



Comparison on dataset COCO



Object detection

Some new architectures for object detection

- Cascade R-CNN – fourth version of R-CNN (2017)
- RefineDet (2017), RefineDet++ (2020)
- FPN – feature pyramid net (2017)
- RetinaNet (2017) – focal loss fcn
- DSSD (2017) – deconvolution SSD
- Efficient Det (2019)
- Mask R-CNN – object detection with segmentation (2017)
- NAS-FPN – Learning Scalable Feature Pyramid Architecture (2019)

Object detection

Elimination of anchors

New trend?

- Object as Point – based on detection of keypoints found by detector utilizing heat maps (called Hourglass network)



- CornerNet (2019) – it uses heat-map of corner boxes => „anchor free
- CenterNet (2020) – uses heat-map peaks to represent object centers, and the network will regress the box width and height directly from these box centers

